



Arnaud Malapert et Marie Pelleau
Université Côte d'Azur, CNRS, I3S, France

Résumé

Et si créer des vidéos des Jeux Olympiques devenait un défi d'algorithmique ?
L'activité Flemme Olympique propose une approche ludique de l'algorithmique à travers un problème de partition. Elle permet d'explorer concrètement des stratégies algorithmiques dans un cadre débranché et de développer la pensée logique.

Table des matières

1	Bienvenue aux Flemmes Olympiques !	3
2	Description du problème	4
3	Partition parfaite des entiers de 1 à n	9
4	Algorithme glouton	11
5	Algorithme glouton répété	15
6	Programmation dynamique	18
7	Citius, altius, fortius	22
8	Problèmes connexes	23
A	Guide pour les animateurs	24
B	Tableau des instances	26
C	Liste des exercices	26

D Licence et reproductibilité	27
Références	27

List of Corrections

Note : Ajouter une section « Description du jeu » avec des photos du jeu. Ajouter aussi des photos des solutions aux exercices obtenues avec le jeu?	3
Warning : Faire attention à la distinction entre sac et sacoche lors de la relecture.	4
Error : Il y a encore des objets avec la même couleurs.	13

1 Bienvenue aux Flemmes Olympiques !

C'est le premier jour de votre stage dans une société de production audiovisuelle pour couvrir les jeux olympiques. Vous rêvez d'assister à des exploits sportifs, d'interviewer les plus grands athlètes dans des enceintes célèbres !

Votre mission est de créer le résumé parfait des Jeux Olympiques, en combinant des séquences de différents sports. Certaines sont courtes, d'autres longues, certaines spectaculaires, d'autres plus calmes... et votre objectif est de tout organiser pour créer deux séquences de durées identiques pour les génériques de début et de fin d'une émission.

Pourquoi cette précision ? Les diffusions reposent sur des serveurs capricieux et des automatismes stricts. Une vidéo trop longue ou trop courte peut bloquer tout le système, interrompre d'autres diffusions, et faire grincer des dents dans les salles de contrôle.

La flemme ! Vous partez la tête basse... adieu les rêves de lumière... et en route pour la salle de montage obscure. Au début, vous tentez d'organiser les séquences à la main, en espérant gagner du temps. Mauvaise idée. Les vidéos s'entrechoquent, certaines séquences débordent, d'autres restent à moitié inutilisées... et vous commencez à désespérer.

Après plusieurs heures de vains efforts, un homme à l'air vif et intelligent se présente à vous : « Richard Ernest Bellman, maître des algorithmes, appelé à la rescousse ! Je vais vous aider à trouver une partition parfaite lorsque c'est possible, et la plupart du temps dans un temps raisonnable, sauf dans les pires cas. »

Et pour vous guider, l'esprit d'Alan Jay Perlis, pionnier de la programmation et des langages informatiques, est là pour distiller ses conseils et réflexions sur les algorithmes... à travers ses fameuses épigrammes [9].

Sa première leçon : simplifiez le problème. Considérez chaque séquence comme un objet réduit à sa seule durée et préparez-vous à appliquer des algorithmes plus ou moins retors selon les situations.

Alan vous avertit cependant que l'efficacité et les gains de temps n'arriveront qu'après un sérieux effort pour comprendre le problème et dompter les algorithmes. Mais une fois ce coup de maître acquis, vous pourrez enfin profiter de votre flemme olympique... sans catastrophe vidéo.

Epigramme d'Alan J. Perlis

On croit comprendre en apprenant, on est plus confiant en écrivant, encore plus en enseignant... mais on en est vraiment sûr en programmant.

1

FiXme Note!

1. FiXme Note: Ajouter une section « Description du jeu » avec des photos du jeu. Ajouter aussi des photos des solutions aux exercices obtenues avec le jeu ?

2 Description du problème

Après la simplification, il faut mettre un peu d'ordre dans tout ça : définir clairement le problème et en explorer les propriétés. Car soyons honnêtes : concevoir, ou même comprendre, un algorithme pour un problème qui nous est étranger, c'est un peu comme écouter une conversation dans une langue étrangère.

Que comprendriez-vous, par exemple, aux conseils d'un marin en pleine régates olympique, si vous n'avez jamais mis un pied sur un bateau ? « Je ne vais pas trop tendre le cunningham ni forcer sur le hale-bas au près ; je vais plutôt jouer sur l'équilibre du bateau en déplaçant mon poids et ajuster finement le cap, vu la petite brise. »

En clair, sans un peu d'expérience de la mer, ces mots ne sont que du vent !

Comme chaque séquence vidéo n'est caractérisée que par sa durée, on peut la considérer comme un simple entier. On introduit ainsi la notion de sac d'entiers, qui regroupe toutes les durées associées aux séquences utilisées pour les génériques de début et de fin.

Définition 2.1 – Sac d'entiers

Un *sac d'entiers* est une collection d'entiers naturels dans laquelle un même nombre peut apparaître plusieurs fois. Par exemple, le sac

$$S = \{1, 2, 2, 5\}$$

contient quatre entiers et le nombre 2 apparaît deux fois.

Plus généralement, un sac S de n entiers naturels peut être noté

$$S = \{s_1, s_2, \dots, s_n\}$$

où chaque s_i est un entier strictement positif.

Définition 2.2 – Partition d'un sac

Une *partition* d'un sac S consiste à répartir les éléments de S en deux sacs non vides S_1 et S_2 , appelés *sacoches*. Chaque élément du sac initial doit appartenir à *exactement une* des deux sacoches.

Autrement dit :

- S_1 et S_2 sont non vides ;
- S_1 et S_2 sont disjoints (aucun entier s_i n'apparaît dans les deux sacs) ;
- S_1 et S_2 recouvrent S (tous les éléments de S apparaissent dans S_1 ou S_2).

Exemple – Partition d'un sac d'entier

Soit un sac $S = \{1, 2, 3, 4, 5\}$.

- Les sacs S_1 et S_2 forment une partition de S .
 - $S_1 = \{1\}$ et $S_2 = \{2, 3, 4, 5\}$
 - $S_1 = \{2, 4\}$ et $S_2 = \{1, 3, 5\}$
- Les sacs S_1 et S_2 ne forment pas une partition de S .
 - $S_1 = \{1, 2, 3, 4, 5\}$ et $S_2 = \emptyset$, car S_2 est vide.
 - $S_1 = \{1, 2, 3\}$ et $S_2 = \{3, 4, 5\}$, car le nombre 3 apparaît dans les deux sacs.
 - $S_1 = \{1, 2\}$ et $S_2 = \{4, 5\}$, car le nombre 3 n'apparaît dans aucun des deux sacs.

Exemple – Partition d'un sac d'entiers avec répétitions

Soit un sac $S = \{1, 2, 2, 5\}$. Les sacs $S_1 = \{2, 2\}$ et $S_2 = \{1, 5\}$ forment une partition de S . La valeur entière 2 apparaît deux fois dans la sacoche S_1 , mais elle correspond à deux entiers distincts : $s_1 = 2$ et $s_2 = 2$.

Dans cette activité, pour simplifier mais sans perte de généralité, nous ne considérerons que des sacs d'entiers sans répétition.

Définition 2.3 – Partition parfaite d'un sac pair

Un sac d'entiers S est dit *pair* si la somme de tous ses entiers est un nombre pair. Une *partition parfaite* d'un sac pair est une partition (S_1, S_2) telle que la somme des entiers de S_1 est égale à la somme des entiers de S_2 .

Exemple – Partition parfaite d'un sac pair

$S = \{1, 2, 3, 4\}$ est un sac pair.

- $S_1 = \{1, 3\}$ et $S_2 = \{2, 4\}$ ne forment pas une partition parfaite, car $1 + 3 \neq 2 + 4$.
- $S_1 = \{1, 4\}$ et $S_2 = \{2, 3\}$ forment une partition parfaite, car $1 + 4 = 2 + 3$.

Exercice 2.1 – Partition de quatre entiers consécutifs (5 min)

Trouver une partition parfaite pour un sac de quatre entiers consécutifs. Par exemple, prenez $S = \{5, 6, 7, 8\}$ ou $S = \{9, 10, 11, 12\}$.

La figure 1 illustre la partition du sac $S = \{5, 6, 7, 8\}$. Chaque ligne correspond à une sacoche. Le tableau de gauche indique les entiers qui composent la sacoche ; les entiers qui n'y appartiennent pas sont barrés. Le tableau de droite représente le remplissage de

chaque sacoché par ces entiers.

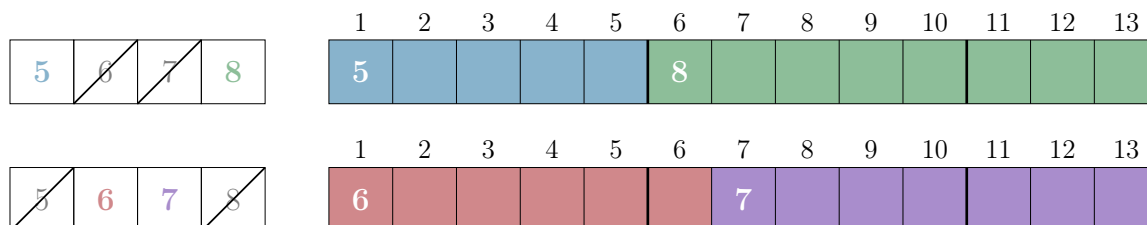


FIGURE 1 – Partition parfaite de quatre entiers consécutifs.

Solution. Partition de quatre entiers consécutifs La partition pour les entiers de 1 à 4 se généralisent pour les sacs de quatre entiers consécutifs. Ce sac $S = \{n+1, n+2, n+3, n+4\}$ est pair ($n \geq 0$).

- $S_1 = \{n+1, n+4\}$, de somme $n+1+n+4 = 2n+5$,
- et $S_2 = \{n+2, n+3\}$, de somme $n+2+n+3 = 2n+5$,
- forment une partition parfaite, car leurs sommes sont égales.

◁

Définition 2.4 – Partition parfaite d'un sac impair

Un sac d'entiers S est dit *impair* si la somme de tous ses entiers est un nombre impair.

Une *partition parfaite* d'un sac impair est une partition (S_1, S_2) telle que la différence entre la somme des entiers de S_1 et celle de S_2 est égale à 1.

En effet, lorsque la somme totale est impaire, il est impossible de partager les entiers en deux sacoches ayant exactement la même somme. La meilleure partition possible est donc celle où les deux sommes diffèrent de 1.

Exemple – Partition parfaite d'un sac impair

Le sac $S = \{1, 2, 3, 4, 5\}$ est impair car $1 + 2 + 3 + 4 + 5 = 15$.

- $S_1 = \{2, 4\}$ et $S_2 = \{1, 3, 5\}$ ne forment pas une partition parfaite, car $2+4 = 6$ et $1+3+5 = 9$.
- $S_1 = \{1, 2, 5\}$ et $S_2 = \{3, 4\}$ forment une partition parfaite, car $1+2+5 = 8$ et $3+4 = 7$, et la différence entre les deux sommes est 1.

Remarque 2.1 – Déterminer la parité d'un sac

Pour savoir si la somme de tous les entiers d'un sac d'entiers est paire ou impaire, il suffit de compter le nombre d'entiers impairs.

- Si le nombre d'entiers impairs est pair, la somme sera paire.
- Si le nombre d'entiers impairs est impair, la somme sera impaire.

Exercice 2.2 – Partition de six entiers consécutifs (5 min)

Trouver une partition parfaite pour un sac de six entiers consécutifs. Par exemple, prenez $S = \{5, 6, 7, 8, 9, 10\}$ ou $S = \{12, 13, 14, 15, 16, 17\}$.

La figure 2 illustre la partition du sac $S = \{5, 6, 7, 8, 9, 10\}$. On peut remarquer que le sac est impair puis les deux sommes diffèrent de 1. En effet, le sac contient trois nombres impairs $\{5, 7, 9\}$.

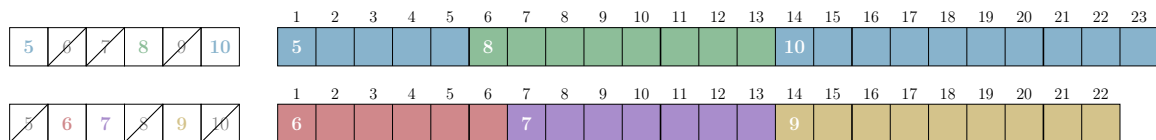


FIGURE 2 – Partition parfaite de six entiers consécutifs.

Solution. Partition de six entiers consécutifs Le sac $S = \{n+1, n+2, n+3, n+4, n+5, n+6\}$ est impair ($n \geq 0$). Sa somme est $n+1+n+2+n+3+n+4+n+5+n+6 = 6n+21$.

- $S_1 = \{n+1, n+4, n+6\}$, de somme $n+1+n+4+n+6 = 3n+11$,
- et $S_2 = \{n+2, n+3, n+5\}$, de somme $n+2+n+3+n+5 = 3n+10$,
- forment une partition parfaite, car la différence entre les deux sommes est 1.

◁

Définition 2.5 – Problème de partition

En informatique, le problème de partition consiste à déterminer si une partition parfaite d'un sac d'entiers existe.

L'idée est simple : il suffit de répartir les entiers entre deux sacoches et de comparer leurs sommes. Calculer la somme des entiers d'une sacoches est une opération facile et rapide.

Définition 2.6 – Complexité du problème de partition

Le nombre de partitions possibles devient très grand dès que le nombre d'entiers augmente. En effet, chaque entier peut être placé dans l'une ou l'autre des deux sacoches : le nombre total de configurations double à chaque nouvel entier ajouté au sac. Ainsi, même si chaque vérification est simple, le nombre total de cas à examiner peut devenir énorme : on parle d'*explosion combinatoire*.

Ce problème est connu pour être difficile en informatique théorique. Plus précisément, il est classé *NP-complet* :

- on peut vérifier rapidement si une partition candidate est correcte ;
- mais trouver une partition parfaite pour n'importe quel sac peut être très difficile.

Remarque 2.2 – Résolution du problème de partition

On distingue deux types d'algorithmes pour résoudre ce problème. Les algorithmes *exacts* déterminent si une partition parfaite existe ou non, sans erreur possible, mais peuvent être coûteux en temps de calcul. Les algorithmes *approximatifs* ou heuristiques, quant à eux, construisent rapidement une solution plausible sans explorer toutes les possibilités, mais ne garantissent pas de trouver une solution même si elle existe.

Dans la pratique, des algorithmes efficaces permettent de résoudre le problème de partition exactement pour des instances de taille raisonnable, et de manière approximative pour des instances de grande taille.

Pour cette raison, on l'appelle parfois :

le plus facile des problèmes difficiles.

On utilise parfois des méthodes approximatives dites heuristiques : elles construisent une solution plausible sans explorer tous les cas possibles. Elles peuvent donc être très rapides, mais elles ne garantissent pas de trouver une solution même si elle existe.

Remarque 2.3 – Symétries du problème

Ce problème admet une symétrie évidente puisque l'on peut inverser la partition, c'est-à-dire échanger les ensembles S_1 et S_2 .

De manière générale, on peut échanger des nombres entre S_1 et S_2 tant que les sommes des deux sacoches restent identiques.

Exemple – Symétries du problème

soit le sac $S = \{1, 2, 4, 6, 8, 9\}$ et la partition parfaite de somme égale à 15.

$$S_1 = \{6, 9\}, \quad S_2 = \{1, 2, 4, 8\}.$$

Inversion de la partition : on peut inverser la partition et les sommes restent identiques.

$$S_1 = \{1, 2, 4, 8\}, \quad S_2 = \{6, 9\}$$

Échange partiel de nombres : on peut aussi échanger des nombres sans changer les sommes, par exemple en remplaçant $6 \in S_1$ par 2, $4 \in S_2$.

$$S_1 = \{2, 4, 9\}, \quad S_2 = \{1, 6, 8\},$$

Exercice 2.3 – Échanges partiels (5 min)

Trouver d'autres partitions parfaites de six entiers consécutifs en effectuant des échanges partiels d'entiers entre les deux sacs.

Epigramme d'Alan J. Perlis

La symétrie permet de réduire la complexité ; cherchez-la partout.

3 Partition parfaite des entiers de 1 à n



Avant de se lancer dans des séquences vidéo dignes des Jeux Olympiques, commençons doucement avec des petites séquences faciles à manipuler : les entiers de 1 à n . Explorez les partitions, essayez, échouez, recommencez.

Exercice 3.1 – Partition parfaite de 1 à n (10 min)

Trouver une partition parfaite des entiers de 1 à n pour $n = 7, 8, 9$.

Remarque 3.1 – Somme des entiers de 1 à n

La somme des entiers de 1 à n vaut

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}.$$

Une anecdote célèbre raconte que le mathématicien allemand Carl Friedrich Gauss aurait découvert cette formule lorsqu’il était encore écolier.

Son professeur lui aurait demandé de calculer la somme des entiers de 1 à 100 afin d’occuper la classe pendant un moment. Mais Gauss remarqua rapidement que l’on pouvait associer les nombres par paires :

$$1 + 100 = 101, \quad 2 + 99 = 101, \quad 3 + 98 = 101, \quad \dots$$

Chaque paire donne la même somme, 101, et il y a 50 paires. La somme vaut donc $50 \times 101 = 5050$.

Ce raisonnement fonctionne pour tout n et conduit à la formule générale $\frac{n(n+1)}{2}$. Cette formule permet notamment de connaître rapidement la somme totale des objets, ce qui sera utile pour déterminer la capacité de la sacoche dans les problèmes de partition.

Solution. La figure 3 présente des partitions parfaites des entiers de 1 à n pour $n = 7, 8, 9$. Chaque ligne correspond à une partition, pour laquelle une sacoche est représentée. L’autre sacoche se déduit facilement : les entiers qui la composent sont barrés dans le tableau de gauche.

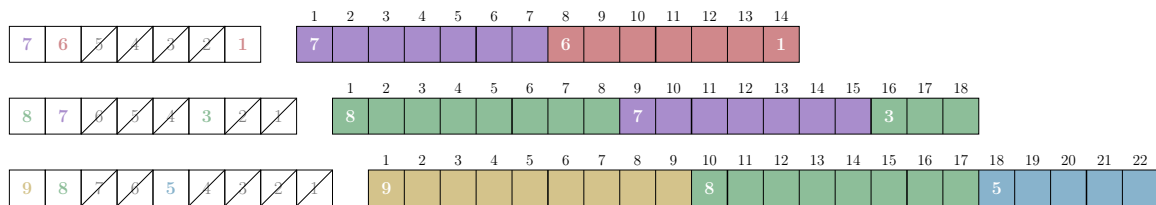


FIGURE 3 – Partition parfaite des entiers de 1 à n pour $n = 7, 8, 9$.

Définition 3.1 – Algorithme

Un *algorithme* est une suite d’étapes simples qui permet de résoudre un problème. Le nombre d’étapes peut varier en fonction de la taille des données.

Remarque 3.2 – Ce que l’activité mettra en évidence

- Plusieurs algorithmes peuvent répondre à un même problème.
- Un algorithme peut répondre à plusieurs problèmes.

Exercice 3.2 – Algorithme pour partition parfaite (20 min)

Proposer un algorithme pour trouver une partition parfaite des entiers de 1 à n .

Un algorithme peut prendre des formes variées : une formule, un pseudo-code, un texte en langue naturelle, ou encore un programme informatique écrit dans un langage de programmation.

Par exemple, on peut le représenter avec deux participants : un participant qui fournit les instructions de résolution (l’algorithme) et un participant qui les exécute pas à pas (le programme), afin de résoudre un problème de partition des entiers de 1 à 12.

Pour éviter toute adaptation en cours d’exécution, le participant qui donne les instructions peut, par exemple, fermer les yeux ou se tourner dos au second participant pendant que celui-ci exécute l’algorithme.

Epigramme d’Alan J. Perlis

Écrire un mauvais programme est facile ; comprendre un bon programme est difficile.

4 Algorithme glouton



Vous avez maintenant compris comment jongler avec les entiers et trouver des partitions parfaites... mais qui a envie de tester toutes les combinaisons possibles ? Un algorithme glouton correspond à la manière la plus intuitive de résoudre le problème pour les débutants. Il construit une solution étape par étape, sans se compliquer la vie. Ce n’est pas toujours une voie optimale, mais c’est simple, rapide et permet de se familiariser avec les algorithmes.

Jusqu’à présent, nous avons manipulé des sacs d’entiers sans leur donner de sens concret. Pour appliquer les algorithmes, nous allons maintenant considérer chaque entier comme la *taille* d’un objet réel, ici la durée d’une séquence vidéo. Ainsi, chaque entier correspond à un objet que l’on peut placer dans une sacoche de capacité donnée. Dans les sections suivantes, nous utiliserons donc le terme *objet* tout en gardant à l’esprit qu’ils sont seulement caractérisés par un entier, leur taille.

Définition 4.1 – Algorithme glouton

Un *algorithme glouton* est un algorithme qui, étape par étape, fait toujours le choix considéré comme le meilleur à ce moment précis (choix *localement optimal*).

Remarque 4.1 – Algorithme glouton

Dans certains cas, un algorithme glouton garantit de trouver une solution si elle existe. Dans le cas général, il s'agit d'une *heuristique* qui construit une solution plausible, sans garantir d'en trouver une, même si elle existe.

Epigramme d'Alan J. Perlis

Si quelqu'un hoche la tête pendant que vous expliquez un algorithme, réveillez-le.

Algorithme 4.1 – Algorithme glouton

- Déterminer la capacité de la sacoche, égale à la moitié de la somme des tailles des objets du sac.
- Trier les objets par ordre décroissant de taille.
- Tant qu'il reste des objets dans le sac :
 - placer le plus grand objet du sac si sa capacité de la sacoche le permet ;
 - Sinon, retirer l'objet du sac ;
 - Si la sacoche est pleine, arrêter l'algorithme.

Dans les exercices suivants, nous donnerons les instances du problème, c'est-à-dire les sacs à partitionner, sous la forme d'un tableau. La première colonne indique l'identifiant du sac. La deuxième colonne donne sa taille, c'est-à-dire le nombre d'objets qu'il contient. La troisième colonne précise les tailles des objets du sac. La dernière colonne indique la capacité de la sacoche dans laquelle placer les objets, égale à la moitié de la somme des tailles des objets. Nous vous donnons directement cette valeur, car nous pensons que vous avez la flemme de calculer la somme. Le type d'une instance est une indication réservée aux animateurs. Vous en devinerez peut-être la signification à la fin de l'activité.

Exercice 4.1 – Algorithme glouton avec type 1 (15 min)

Id	Taille	Entiers	Capacité
1	6	25, 22, 14, 12, 9, 4	43
2	6	25, 20, 18, 11, 4, 2	40
3	9	21, 17, 15, 13, 9, 6, 5, 4, 2	46
4	9	24, 18, 15, 14, 13, 7, 6, 2, 1	50
5	12	25, 23, 22, 20, 13, 11, 9, 7, 6, 5, 3, 2	73
6	15	25, 23, 22, 20, 19, 18, 16, 15, 12, 11, 8, 5, 3, 2, 1	100

TABLE 1 – Appliquer l'algorithme glouton sur une instance de type 1.

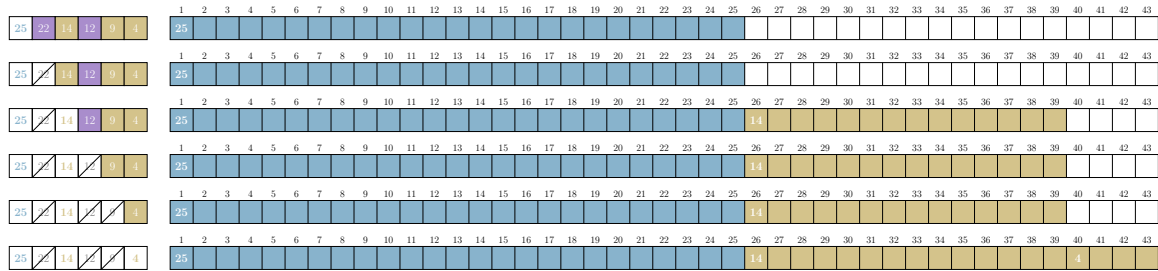


FIGURE 4 – Application de l'algorithme glouton sur l'instance #1 : $S_1 = \{25, 14, 4\}$.

La figure 4 montre la construction de la solution par l'algorithme glouton. À chaque itération, dans le tableau de gauche, les objets placés dans la sacoche apparaissent sur fond blanc, ceux retirés du sac sont barrés, et ceux qui n'ont pas encore été examinés apparaissent sur fond coloré. La partition obtenue à la sixième itération est parfaite puisque la sacoche est pleine. ³

FiXme Error!

Remarque 4.2 – Nombre d'étapes de l'algorithme glouton

Pour un ordre donné des objets, l'algorithme glouton est très rapide :

- Chaque objet est examiné au plus une fois pour être placé dans la sacoche ;
- Le nombre d'étapes est donc proportionnel au nombre n d'objets.

En pratique, même pour plusieurs milliers d'objets, le glouton trouve une partition presque sans transpirer.

3. FiXme Error: Il y a encore des objets avec la même couleurs.

Epigramme d'Alan J. Perlis

La complexité ne se communique pas, elle se pressent.

Exercice 4.2 – Algorithme glouton avec type 2 (15 min)

Id	Taille	Entiers	Capacité
7	6	22, 20, 15, 14, 9, 3	41
8	6	23, 20, 18, 17, 12, 10	50
9	9	22, 20, 17, 13, 9, 6, 5, 4, 2	49
10	9	24, 20, 15, 10, 8, 7, 6, 4, 1	47
11	12	25, 23, 18, 16, 15, 12, 11, 9, 7, 6, 4, 3	74
12	15	25, 23, 21, 19, 18, 17, 15, 14, 13, 10, 7, 6, 5, 4, 3	100

TABLE 2 – Appliquer l'algorithme glouton sur une instance de type 2.

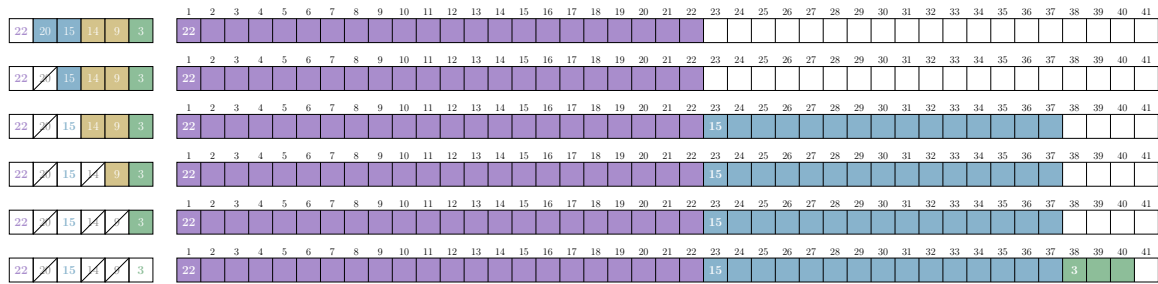


FIGURE 5 – Application de l'algorithme glouton sur l'instance #7 : $S_1 = \{22, 15, 3\}$.

La figure 5 montre la construction de la solution par l'algorithme glouton. L'algorithme examine tous les objets, mais la partition obtenue n'est pas parfaite.

Remarque 4.3 – À propos de l'algorithme glouton

- On peut obtenir d'autres solutions en modifiant l'ordre dans lequel les objets sont considérés.
- Des ordres différents donnent parfois la même solution.
- Répéter l'algorithme glouton avec plusieurs ordres différents permet de générer plusieurs solutions.

Exercice 4.3 – Partition de 1 à n avec le glouton (10 min)

Appliquer l'algorithme glouton sur le sac des entiers de 1 à $n = 15, 16$.

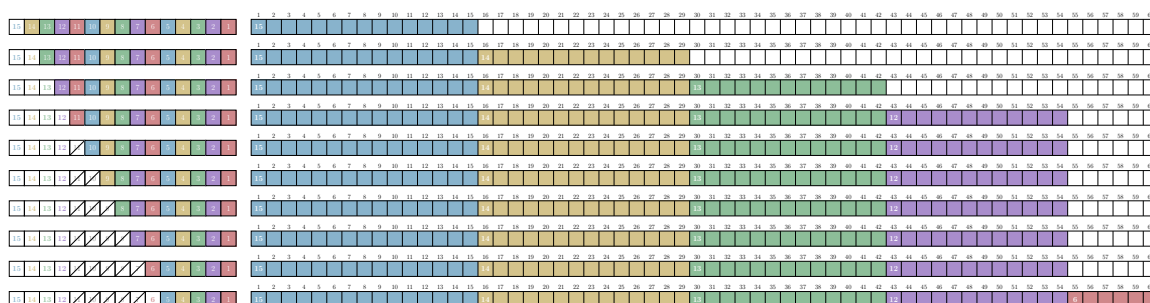


FIGURE 6 – Application de l'algorithme glouton pour la Partition des entiers de 1 à 15 : $S_1 = \{15, 14, 13, 12, 6\}$.

La figure 6 montre la partition parfaite des entiers de 1 à 15 par l'algorithme glouton. L'algorithme place les 4 plus grands objets dans le sac puis un seul objet plus petit pour remplir exactement la sacoche.

Remarque 4.4 – Optimalité du glouton pour la partition des entiers de 1 à n

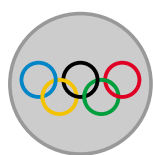
L'algorithme glouton est optimal pour le problème de partition des entiers de 1 à n :

- il existe toujours une partition parfaite des entiers de 1 à n ;
- l'algorithme glouton en trouve toujours une.

Preuve. Supposons qu'à une étape de l'algorithme on ne puisse pas placer l'entier k dans la sacoche. Dans ce cas, la capacité restante de la sacoche est égale inférieure à k .

Or les entiers encore disponibles sont exactement $1, 2, \dots, k$. Il reste donc toujours un entier égal à cette capacité restante, que l'on peut placer dans la sacoche. L'algorithme peut donc toujours compléter le sac.

5 Algorithme glouton répété



L'algorithme glouton semblait prometteur, mais il passe parfois juste à côté d'une partition parfaite. Et pourtant, en effectuant quelques échanges entre les objets, la solution finit par apparaître.

Alan J. Perlis vous glisserait volontiers qu'un programme devient souvent plus intéressant lorsqu'on le relance plusieurs fois en modifiant les conditions initiales. En les faisant varier, on explore des chemins de construction différents.

L'idée est donc simple : si l'algorithme glouton échoue, on recommence en modifiant intelligemment ces conditions initiales départ. C'est précisément le principe de l'algorithme glouton répété : exécuter plusieurs fois le glouton en faisant varier les objets disponibles dans le sac. Concrètement, à chaque itération, on retire du sac le plus grand objet précédemment placé dans la sacoche.

Epigramme d'Alan J. Perlis

Si l'on écrivait des programmes dès l'enfance, on saurait probablement mieux les lire une fois adulte.

Algorithme 5.1 – Algorithme glouton répété

Répéter tant que tous les objets ne sont pas dans la sacoche :

- Appliquer l'algorithme glouton.
- Si la sacoche est pleine, arrêter l'algorithme.
- Sinon, retirer le plus grand objet du sac et recommencer.

Exercice 5.1 – Algorithme glouton répété avec type 2 (15 min)

Id	Taille	Entiers	Capacité
7	6	22, 20, 15, 14, 9, 3	41
8	6	23, 20, 18, 17, 12, 10	50
9	9	22, 20, 17, 13, 9, 6, 5, 4, 2	49
10	9	24, 20, 15, 10, 8, 7, 6, 4, 1	47
11	12	25, 23, 18, 16, 15, 12, 11, 9, 7, 6, 4, 3	74
12	15	25, 23, 21, 19, 18, 17, 15, 14, 13, 10, 7, 6, 5, 4, 3	100

TABLE 3 – Appliquer l'algorithme glouton répété sur une instance de type 2.

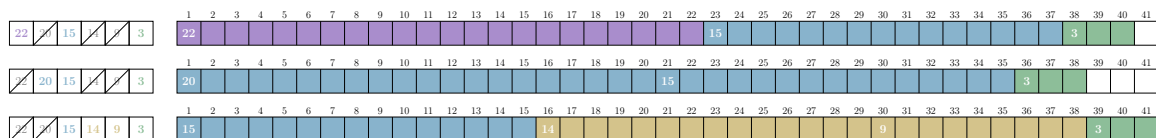


FIGURE 7 – Application de l'algorithme glouton répété sur l'instance #7 : $S_1 = \{15, 14, 9, 3\}$.

La figure 7 montre les différentes solutions obtenues à chaque itération par l'algorithme glouton répété. La partition obtenue à la troisième itération est parfaite.

Remarque 5.1 – À propos de l'algorithme glouton répété

L'algorithme glouton répété peut s'arrêter dès que tous les objets sont dans la sacoche, car retirer des objets ne permettrait pas d'améliorer la solution : la sacoche serait forcément moins remplie.

L'algorithme glouton répété examine uniquement des solutions distinctes : à chaque itération, il retire le plus grand objet dans le sac restant avant de relancer le glouton. Or, cet objet est toujours inclus dans la sacoche construite par le glouton. En effet, c'est le premier objet examiné par le glouton et il est donc toujours placé dans la sacoche.

Remarque 5.2 – Nombre d'étapes de l'algorithme glouton répété

- L'algorithme glouton répété explore au plus n solutions (une par objet successivement éliminé, à chaque itération).
- À chaque itération, l'algorithme glouton examine au plus les n objets.
- Le nombre total d'étapes de l'algorithme glouton répété est donc proportionnel à $n^2 = n \times n$.

Exercice 5.2 – Algorithme glouton répété avec type 3 (20 min)

Id	Taille	Entiers	Capacité
13	6	25, 20, 17, 14, 11, 8	47
14	6	23, 20, 17, 13, 12, 5	45
15	9	22, 19, 15, 14, 12, 10, 4, 3, 1	50
16	9	25, 19, 13, 12, 10, 8, 7, 3, 1	49
17	12	20, 17, 16, 14, 13, 12, 11, 10, 9, 6, 5, 3	68
18	15	25, 22, 20, 19, 18, 15, 14, 13, 11, 10, 9, 7, 6, 5, 2	98

TABLE 4 – Appliquer l'algorithme glouton répété sur une instance de type 3.

La figure 8 présente les solutions obtenues par l'algorithme glouton répété. L'algorithme ne trouve pas de partition parfaite. On peut toutefois remarquer que les deux premières itérations produisent les partitions les plus équilibrées, tandis que la dernière itération donne la partition la moins équilibrée, avec une différence de sommes maximale.

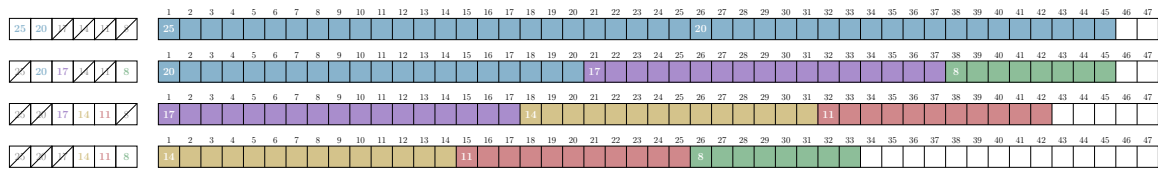


FIGURE 8 – Application de l'algorithme glouton répété sur l'instance #13 : $S_1 = \{14, 11, 8\}$.

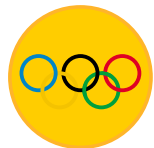
Remarque 5.3 – Ne pas perdre la meilleure solution

En pratique, il est important de garder en mémoire la meilleure solution trouvée par l'algorithme glouton répété, car ce n'est pas forcément la dernière solution calculée qui est la meilleure.

Remarque 5.4 – Recherche locale

Les méthodes de recherche locale reposent sur la répétition d'algorithmes gloutons, combinée à des modifications des solutions obtenues, par exemple par des échanges partiels d'objets.

6 Programmation dynamique



Cette fois, vous avez tout essayé. Le glouton répété n'a pas trouvé de partition parfaite et vos tentatives d'échanges sont restées vaines ? Malgré tous vos efforts la partition parfaite reste introuvable ou peut-être simplement bien cachée ? Malheureusement, votre patron exige une réponse claire : la partition parfaite existe-t-elle ou non ?

C'est alors que Richard Bellman intervient avec une idée plus méthodique. Plutôt que d'explorer les possibilités un peu au hasard, il propose de construire progressivement toutes les sommes que l'on peut atteindre avec les objets disponibles. La méthode est plus coûteuse en temps, mais aussi plus difficile à imaginer et à mettre en œuvre. Il va falloir être rusé ...

Epigramme d'Alan J. Perlis

Pour comprendre un programme, il faut se mettre dans la peau de la machine et du programme.

Algorithme 6.1 – Algorithme de programmation dynamique

- Déterminer la capacité de la sacoche.
- Trier les objets par ordre décroissant.
- Marquer la case 0, qui correspond au point de départ.
- Répéter tant qu’il reste des objets dans le sac :
 - Prendre le plus grand objet du sac.
 - Pour chaque case marquée, en partant de la dernière (à droite) :
 - Calculer la case atteinte en plaçant immédiatement l’objet après la case marquée.
 - Si cette case n’est pas encore marquée, y placer un marqueur correspondant à l’objet.
 - Retirer l’objet du sac.
- Si la dernière case (capacité) est marquée, le sac est plein et l’algorithme s’arrête.

Nous savons désormais quelles sommes sont atteignables, ce qui est déjà une bonne nouvelle. En revanche, les solutions elles-mêmes restent cachées : à nous de les reconstruire en interprétant les marqueurs, avec l’algorithme comme guide.

Algorithme 6.2 – Reconstruction de la sacoche en programmation dynamique

Tant qu’il reste des marqueurs :

- Sélectionner le marqueur le plus à droite.
- Placer l’objet correspondant à ce marqueur dans la sacoche en retirant les marqueurs situés au dessus-de l’objet.

Exercice 6.1 – Programmation dynamique avec type 3 (20 min)

Id	Taille	Entiers	Capacité
13	6	25, 20, 17, 14, 11, 8	47
14	6	23, 20, 17, 13, 12, 5	45
15	9	22, 19, 15, 14, 12, 10, 4, 3, 1	50
16	9	25, 19, 13, 12, 10, 8, 7, 3, 1	49
17	12	20, 17, 16, 14, 13, 12, 11, 10, 9, 6, 5, 3	68
18	15	25, 22, 20, 19, 18, 15, 14, 13, 11, 10, 9, 7, 6, 5, 2	98

TABLE 5 – Appliquer la programmation dynamique sur une instance de type 3.

La figure 9 illustre la programmation dynamique. La première ligne donne la sacoche

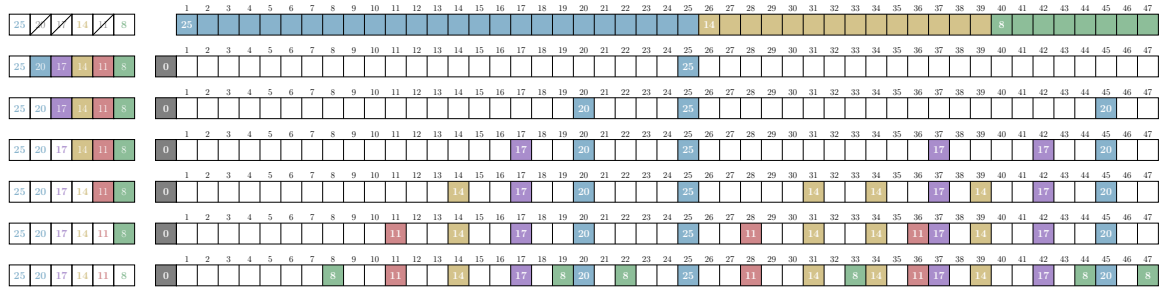


FIGURE 9 – Application de la programmation dynamique sur l'instance #13 : $S_1 = \{25, 14, 8\}$.

obtenue par reconstruction. Les lignes suivantes correspondent aux étapes de marquage des cases avec les objets.

Après le marquage du point de départ, une seule case est marquée par l'objet 25 lors de la première itération. Deux cases sont marquées avec l'objet 20 lors de la seconde itération. On remarque que seulement trois cases sont marquées par l'objet 11 à la quatrième itération, certaines cases atteintes étant déjà marquées.

Remarque 6.1 – Nombre d'étapes de la programmation dynamique

L'algorithme de programmation dynamique est beaucoup plus gourmand en étapes que les gloutons. Pour n objets et une capacité C de la sacoche :

- La programmation dynamique marque les cases avec un objet au plus n fois (une fois par objet successivement examiné, à chaque itération).
- À chaque itération, il faut examiner toutes les cases marquées, et il y en a au plus C .
- Le nombre d'étapes est donc proportionnel à $n \cdot C$, ce qui est bien plus élevé que pour les gloutons.

Cette complexité ne dépend pas seulement de la taille de l'entrée (le nombre d'objets), mais aussi de leurs valeurs (ici la capacité de la sacoche). Pour une taille et des valeurs raisonnables, l'algorithme reste rapide, mais pour de grandes tailles et valeurs, le temps de calcul peut devenir important.

Exercice 6.2 – Programmation dynamique avec type 4 (25 min)

Id	Taille	Entiers	Capacité
19	6	24, 21, 19, 15, 9, 4	46
20	6	25, 22, 15, 13, 9, 1	42
21	9	25, 23, 18, 15, 13, 10, 8, 5, 3	60
22	9	25, 24, 15, 14, 13, 12, 11, 2, 1	58
23	10	24, 22, 20, 18, 16, 14, 12, 10, 6, 4	73
24	11	24, 22, 20, 18, 16, 14, 12, 10, 8, 6, 4	77

TABLE 6 – Appliquer la programmation dynamique sur une instance de type 4.

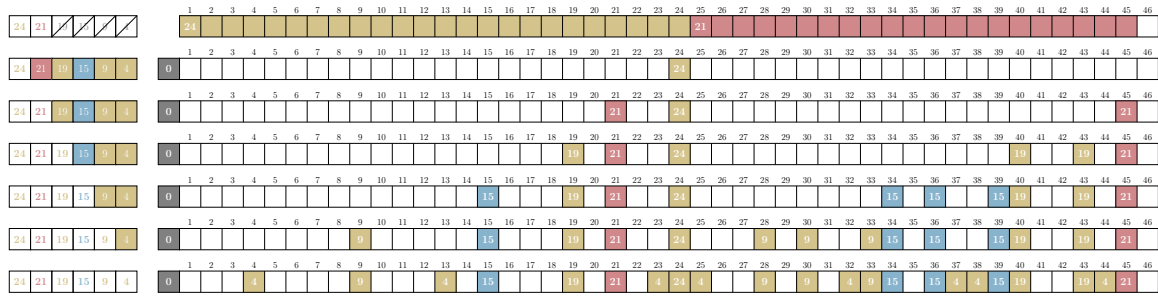


FIGURE 10 – Application de la programmation dynamique sur l'instance #19 : $S_1 = \{24, 21\}$.

La figure 10 illustre la programmation dynamique lorsqu'il n'existe pas de partition parfaite. La reconstruction de la solution à partir de la case marquée la plus à droite permet d'obtenir la sacoche la plus remplie, et donc la partition la plus équilibrée possible.

Remarque 6.2 – Optimalité de la programmation dynamique

L'algorithme de programmation dynamique garantit de trouver une partition la plus équilibrée :

- il trouve une partition parfaite si elle existe ;
- sinon, il minimise la différence entre les sommes des deux sacs.

On sacrifie la rapidité pour être sûr d'obtenir une solution optimale.

Remarque 6.3 – Une ouverture en profondeur de Bellman

Il existe une méthode exacte plus générale pour résoudre ce type de problème : *Tester-et-Générer*. Simple à comprendre et quelquefois facile à programmer, elle devient vite très pénible à exécuter à la main, tant elle explore toutes les possibilités par sauts de puce. Après avoir déjà transpiré sur la programmation dynamique, notre patience a des limites... et nous n'irons pas plus loin dans cette voie !

Remarque 6.4 – Bellman botte en touche !

Il existe aussi une pléthore de méthodes approximatives plus élaborées pour s'échapper des limites des algorithmes gloutons. Elles reposent souvent sur des échanges d'objets et s'inspirent librement de phénomènes naturels : évolution génétique, colonies de fourmis, essaims d'abeilles, etc.

Mais Richard Bellman rechigne à vous en dire davantage : selon lui, c'est déjà bien suffisant pour aujourd'hui et pour résoudre le problème ... il a la flemme.

7 Citius, altius, fortius⁴

Votre esprit est maintenant limpide ! Vous voilà initié aux arcanes des partitions. Mais une question demeure : comment ce labeur acharné peut-il se transformer en véritable flemme olympique ?

C'est alors qu'Alan, malicieusement, vous souffle à nouveau à l'oreille :

Epigramme d'Alan J. Perlis

On croit comprendre en apprenant, on est plus confiant en écrivant, encore plus en enseignant... mais on en est vraiment sûr en programmant.

Exercice 7.1 – Olympiade de partition (? min)

Écrire un programme pour résoudre le problème de partition et le soumettre au juge olympique (**Bientôt disponible !**).

À ce prix seulement, vous pourrez espérer rejoindre le panthéon des partitions. Et qui sait ? Peut-être pourrez-vous enfin profiter des Jeux... autrement que derrière un écran.

4. Plus vite, plus haut, plus fort.

Epigramme d'Alan J. Perlis

Pensez à toute l'énergie mentale dépensée pour tenter de tracer la frontière entre « algorithme » et « programme ».

8 Problèmes connexes

Le problème de partition appartient à une grande famille de problèmes d'optimisation étudiés en informatique et en recherche opérationnelle. La recherche opérationnelle utilise des modèles mathématiques et des méthodes algorithmiques, implémentées en informatique, pour résoudre des problèmes concrets de décision, comme l'organisation, la planification ou l'allocation de ressources. De nombreux problèmes célèbres lui sont étroitement liés.

Epigramme d'Alan J. Perlis

Tout programme fait au moins deux choses : ce pour quoi il a été conçu ... et une autre pour laquelle il ne l'a pas été.

Définition 8.1 – Problème de la somme de sous-ensembles

On dispose d'un sac d'entiers et d'une capacité donnée. La question est de savoir s'il existe une sacoche d'entiers dont la somme est exactement égale à cette capacité. Le problème de partition en est un cas particulier où la capacité est la moitié de la somme totale des entiers.

Par exemple, la sélection d'investissements sous contrainte de budget, en choisissant des projets dont le coût total respecte un budget donné. Ou encore, le choix d'activités à réaliser dans un temps limité, en optimisant l'utilisation du temps disponible.

Définition 8.2 – Problème d'ordonnancement sur deux machines

Un problème très proche apparaît en *ordonnancement*. Supposons que plusieurs tâches doivent être réparties sur deux machines identiques. Chaque tâche a une durée d'exécution. On cherche à minimiser le délai total, c'est-à-dire la date de fin de la dernière tâche. Pour cela, il faut répartir les tâches entre les deux machines de manière à équilibrer au mieux leurs durées totales. Ce problème revient donc à chercher une partition des durées des tâches aussi équilibrée que possible.

Ce problème apparaît par exemple dans la répartition de travaux entre deux serveurs informatiques, dans l'organisation de tâches de production sur deux chaînes d'assemblage,

ou encore dans la planification de devoirs à réaliser entre deux jours de travail.

Remarque 8.1 – Un algorithme peut répondre à plusieurs problème !

Tous les algorithmes introduits fonctionnent pour la somme des sous-ensembles et l'ordonnancement sur deux machines.

Définition 8.3 – Problème du sac à dos

Chaque objet possède une taille et une valeur. Le but est de choisir des objets à placer dans un sac de capacité limitée de manière à maximiser la valeur totale. Contrairement au problème de partition, il ne s'agit plus de trouver une somme exacte, mais de sélectionner la meilleure combinaison possible en termes de valeurs.

Ce problème apparaît par exemple en industrie, lors du choix de composants à produire sous contrainte de capacité (temps, masse, ou coût), chaque élément ayant à la fois une contrainte de consommation de ressources et une valeur associée à son intérêt économique. L'objectif est alors de sélectionner les éléments les plus rentables sans dépasser les ressources disponibles.

Définition 8.4 – Problème de mise en boîtes

On dispose de plusieurs sacs de capacité fixe et d'objets de tailles différentes. Le but est de ranger tous les objets en utilisant le moins de sacs possible. Ce problème généralise l'idée de remplir une sacs avec des objets de tailles données.

Ce problème apparaît par exemple dans le transport routier, maritime ou aérien, où des marchandises de volumes différents doivent être réparties dans un nombre minimal de camions, conteneurs ou soutes de capacité fixe, sans dépasser leur capacité.

Au delà de ces quelques exemples, ces problèmes [1–5] apparaissent dans de nombreux domaines : logistique, planification de tâches, allocation de ressources, gestion de données, télécommunications ou encore industrie. Ils illustrent l'importance des algorithmes pour organiser et optimiser des ressources limitées.

Epigramme d'Alan J. Perlis

La simplicité ne précède pas la complexité, elle la suit.

A Guide pour les animateurs

L'activité Flemme Olympique s'inscrit dans une démarche pédagogique visant à rendre accessibles des notions fondamentales de l'algorithmique et de l'optimisation à travers une situation concrète. Inspirée de la problématique de la diffusion de contenus audiovisuels,

elle invite les participants et participantes à relever un défi : construire un résumé parfaitement équilibré en combinant des séquences de durées variées. Derrière cette mise en situation ludique se cache un problème classique de l'informatique théorique, celui du partitionnement d'un ensemble d'entiers.

Dans un premier temps, les participants explorent le problème de manière intuitive. Ils manipulent des données simples (durées de séquences) et tentent, par essais et erreurs, de construire deux ensembles équilibrés. Cette phase favorise l'engagement, la prise d'initiative et la compréhension progressive des contraintes, notamment la notion de parité et d'équilibre des sommes. Rapidement, les limites de cette approche empirique apparaissent, ouvrant la voie à une réflexion plus structurée.

L'activité introduit alors différentes stratégies algorithmiques. L'algorithme glouton, accessible et intuitif, constitue une première formalisation de la démarche de résolution. Il permet de comprendre comment une suite de choix locaux peut conduire à une solution satisfaisante, sans garantir l'optimalité. Dans un second temps, l'algorithme glouton répété illustre l'intérêt de la diversification des essais et de la variation des conditions initiales. Enfin, la programmation dynamique propose une approche plus systématique et rigoureuse, garantissant l'obtention d'une solution optimale lorsque celle-ci existe, au prix d'une complexité accrue.

Au-delà des contenus disciplinaires, cette activité mobilise des compétences transversales essentielles. Elle développe la pensée algorithmique, la capacité à modéliser une situation réelle, à raisonner de manière logique et à comparer différentes stratégies. Elle favorise également la collaboration, la communication et l'argumentation, notamment lors des phases de mise en commun et de justification des solutions.

D'un point de vue didactique, Flemme Olympique illustre l'intérêt des activités débranchées pour introduire des concepts informatiques complexes sans recourir immédiatement à la programmation. Elle permet aux participants de se concentrer sur la compréhension des mécanismes sous-jacents, tout en maintenant un haut niveau d'engagement. Par ailleurs, elle offre une ouverture vers des notions avancées telles que la complexité algorithmique et les problèmes NP-complets, contribuant ainsi à une acculturation progressive aux enjeux de l'informatique contemporaine et de la recherche opérationnelle.

En somme, cette activité constitue un levier efficace pour articuler manipulation, abstraction et formalisation, tout en développant des compétences clés du XXI^e siècle dans un cadre motivant et accessible.

B Tableau des instances

Id	Taille	Entiers	Capacité	Type
1	6	25, 22, 14, 12, 9, 4	43	1
2	6	25, 20, 18, 11, 4, 2	40	1
3	9	21, 17, 15, 13, 9, 6, 5, 4, 2	46	1
4	9	24, 18, 15, 14, 13, 7, 6, 2, 1	50	1
5	12	25, 23, 22, 20, 13, 11, 9, 7, 6, 5, 3, 2	73	1
6	15	25, 23, 22, 20, 19, 18, 16, 15, 12, 11, 8, 5, 3, 2, 1	100	1
7	6	22, 20, 15, 14, 9, 3	41	2
8	6	23, 20, 18, 17, 12, 10	50	2
9	9	22, 20, 17, 13, 9, 6, 5, 4, 2	49	2
10	9	24, 20, 15, 10, 8, 7, 6, 4, 1	47	2
11	12	25, 23, 18, 16, 15, 12, 11, 9, 7, 6, 4, 3	74	2
12	15	25, 23, 21, 19, 18, 17, 15, 14, 13, 10, 7, 6, 5, 4, 3	100	2
13	6	25, 20, 17, 14, 11, 8	47	3
14	6	23, 20, 17, 13, 12, 5	45	3
15	9	22, 19, 15, 14, 12, 10, 4, 3, 1	50	3
16	9	25, 19, 13, 12, 10, 8, 7, 3, 1	49	3
17	12	20, 17, 16, 14, 13, 12, 11, 10, 9, 6, 5, 3	68	3
18	15	25, 22, 20, 19, 18, 15, 14, 13, 11, 10, 9, 7, 6, 5, 2	98	3
19	6	24, 21, 19, 15, 9, 4	46	4
20	6	25, 22, 15, 13, 9, 1	42	4
21	9	25, 23, 18, 15, 13, 10, 8, 5, 3	60	4
22	9	25, 24, 15, 14, 13, 12, 11, 2, 1	58	4
23	10	24, 22, 20, 18, 16, 14, 12, 10, 6, 4	73	4
24	11	24, 22, 20, 18, 16, 14, 12, 10, 8, 6, 4	77	4

C Liste des exercices

2.1	Partition de quatre entiers consécutifs (5 min)	5
2.2	Partition de six entiers consécutifs (5 min)	7
2.3	Échanges partiels (5 min)	9
3.1	Partition parfaite de 1 à n (10 min)	9
3.2	Algorithme pour partition parfaite (20 min)	11
4.1	Algorithme glouton avec type 1 (15 min)	13
4.2	Algorithme glouton avec type 2 (15 min)	14
4.3	Partition de 1 à n avec le glouton (10 min)	15

5.1	Algorithme glouton répété avec type 2 (15 min)	16
5.2	Algorithme glouton répété avec type 3 (20 min)	17
6.1	Programmation dynamique avec type 3 (20 min)	19
6.2	Programmation dynamique avec type 4 (25 min)	21
7.1	Olympiade de partition (? min)	22

D Licence et reproductibilité

Cette activité est libre, ouverte, et reproductible dans le respect des licences. Ainsi, toute personne peut vérifier, reproduire et prolonger les résultats proposés.

<https://github.com/arnaud-m/flemolymp>

Elle contient notamment :

- le code source et les données sous la forme d’une extension R, distribué sous licence MIT ;
- le contenu pédagogique (textes, images, schémas, diagrammes) distribué sous licence CC BY 4.0.
- le source L^AT_EX de tous les documents (manuel, fiche technique, solutions) ;
- les fichiers nécessaires à la fabrication des éléments physiques via découpeuse laser ou imprimante 3D ;
- les illustrations et images utilisées dans l’activité, libres de droits ou sous licence compatible ;
- les instructions détaillées pour la mise en place de l’activité, incluant la préparation du matériel et des variantes selon le niveau des participants.

Références

- [1] Problème de mise en boîtes (bin packing problem). https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_bin_packing.
- [2] Problème de sac-à-dos (knapsack problem). https://fr.wikipedia.org/wiki/Probl%C3%A8me_du_sac_%C3%A0_dos.
- [3] Problème d’ordonnancement avec des machines identiques (identical-machines scheduling). https://en.wikipedia.org/wiki/Identical-machines_scheduling.
- [4] Problème de partition (partition problem). https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_partition.
- [5] Problème de la somme des sous-ensembles (subset sum problem). https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_la_somme_de_sous-ensembles.

- [6] maix. Images de médailles de bronze, argent, ou or disponibles sur wikimedia commons. <https://commons.wikimedia.org/w/index.php?curid=1759963>, 2007. CC BY-SA 2.5.
- [7] Sylvano Martello and Paolo Toth. *Knapsack Problems*. Wiley, 1990.
- [8] Stephan Mertens. The easiest hard problem : Number partitioning, 2003. URL <https://arxiv.org/abs/cond-mat/0310317>.
- [9] Alan J. Perlis. Special feature : Epigrams on programming. *SIGPLAN Not.*, 17 (9) :7–13, September 1982. ISSN 0362-1340. doi : 10.1145/947955.1083808. URL <https://doi.org/10.1145/947955.1083808>.